

K L Deemed to be University

Department of Computer Science and Engineering

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Skilling Session 1: RDBMS Foundations

Name	Chakrika Panakala
Roll No.	2520090042
Course	25CS1302E
CO Mapping	CO1
Session	Skilling 1

Aim

To study the role of a DBMS over a file system, the RDBMS architecture and the three-schema (ANSI/SPARC) model, demonstrate logical and physical data independence, explore the system catalog and data dictionary in PostgreSQL, and audit the Task Hub frontend mock data into database entities (PBL).

Program

The practical uses PostgreSQL to convert the Task Hub application's mock frontend data into relational entities. The database contains users, projects and tasks, with primary keys, foreign keys, validation constraints and views. The script also demonstrates the external, conceptual and internal schema levels, data independence, and PostgreSQL catalog information.

1. Database and Entity Creation

```
DROP DATABASE IF EXISTS taskhub_db;
CREATE DATABASE taskhub_db;
\c taskhub_db

CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    role VARCHAR(10) NOT NULL DEFAULT 'MEMBER'
    CHECK (role IN ('ADMIN', 'MEMBER'))
);

CREATE TABLE projects (
    project_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    owner_id INT NOT NULL REFERENCES users(user_id),
    deadline DATE
);

CREATE TABLE tasks (
    task_id SERIAL PRIMARY KEY,
    project_id INT NOT NULL
    REFERENCES projects(project_id) ON DELETE CASCADE,
    title VARCHAR(200) NOT NULL,
    status VARCHAR(12) NOT NULL DEFAULT 'TODO'
    CHECK (status IN ('TODO', 'IN_PROGRESS', 'DONE')),
    assignee_id INT REFERENCES users(user_id),
    due_date DATE
);
```

2. Sample Task Hub Data

```
INSERT INTO users (full_name, email, role) VALUES
('Habeeb', 'habeeb@taskhub.dev', 'ADMIN'),
('Ananya Rao', 'ananya@taskhub.dev', 'MEMBER'),
('Rahul Verma', 'rahul@taskhub.dev', 'MEMBER');

INSERT INTO projects (name, owner_id, deadline) VALUES
('Task Hub Backend', 1, '2026-11-30'),
('Task Hub Frontend', 2, '2026-12-15');

INSERT INTO tasks (project_id, title, status, assignee_id, due_date) VALUES
```

```
(1, 'Design ER diagram', 'DONE', 1, '2026-10-05'),
(1, 'Create PostgreSQL schema', 'IN_PROGRESS', 2, '2026-10-10'),
(1, 'Write FastAPI CRUD', 'TODO', 3, '2026-10-20'),
(2, 'Replace mock data', 'TODO', 2, '2026-10-25');
```

3. External Schema / Views

```
CREATE VIEW member_task_view AS
SELECT u.full_name AS assignee, t.title, t.status, t.due_date
FROM tasks t
JOIN users u ON u.user_id = t.assignee_id;

CREATE VIEW project_progress_view AS
SELECT p.name AS project,
       COUNT(t.task_id) AS total_tasks,
       COUNT(t.task_id) FILTER (WHERE t.status = 'DONE') AS done_tasks
FROM projects p
LEFT JOIN tasks t ON t.project_id = p.project_id
GROUP BY p.name;

SELECT * FROM member_task_view
WHERE assignee = 'Ananya Rao';

SELECT * FROM project_progress_view
ORDER BY project;
```

4. Internal Schema / Physical Storage

```
SELECT relname AS table_name,
       pg_relation_filepath(oid::regclass) AS data_file,
       pg_size_pretty(pg_relation_size(oid)) AS size_on_disk
FROM pg_class
WHERE relname IN ('users', 'projects', 'tasks')
ORDER BY relname;
```

5. Logical Data Independence

```
ALTER TABLE tasks
ADD COLUMN priority VARCHAR(6) DEFAULT 'MEDIUM';

SELECT * FROM member_task_view
WHERE assignee = 'Ananya Rao';
```

6. Physical Data Independence

```
SET enable_seqscan = off;

EXPLAIN
SELECT title FROM tasks
WHERE assignee_id = 2;

CREATE INDEX idx_tasks_assignee
ON tasks (assignee_id);

EXPLAIN
SELECT title FROM tasks
WHERE assignee_id = 2;

SELECT title FROM tasks
WHERE assignee_id = 2;

RESET enable_seqscan;
```

7. System Catalog and Data Dictionary

```
SELECT table_name, table_type
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_type, table_name;

SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_name = 'tasks'
ORDER BY ordinal_position;

SELECT conname AS constraint_name,
       CASE contype
         WHEN 'p' THEN 'PRIMARY KEY'
         WHEN 'f' THEN 'FOREIGN KEY'
         WHEN 'u' THEN 'UNIQUE'
         WHEN 'c' THEN 'CHECK'
       END AS type,
       conrelid::regclass AS table_name
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace
ORDER BY table_name, type;

SELECT indexname, tablename
FROM pg_indexes
WHERE schemaname = 'public'
```

ORDER BY tablename;

\dt
\dv

Output

The following output is based on the supplied Skill 1 practical. Generated file paths, storage sizes and query-plan costs can vary between PostgreSQL installations.

Database and tables

The database taskhub_db was created successfully, followed by the users, projects and tasks tables.

Inserted records

3 users, 2 projects and 4 tasks were inserted successfully.

Member task view

For Ananya Rao, the view displayed: Create PostgreSQL schema (IN_PROGRESS, 2026-10-10) and Replace mock data (TODO, 2026-10-25).

Project progress view

Task Hub Backend contained 3 tasks with 1 completed task; Task Hub Frontend contained 1 task with 0 completed tasks.

Physical storage

PostgreSQL reported physical relation files for projects, tasks and users, each shown as 8192 bytes in the supplied run.

Logical independence

After adding the priority column to tasks, the existing member_task_view continued to work without modification.

Physical independence

Before the index, the query plan used a Seq Scan. After creating idx_tasks_assignee, the same query used an Index Scan.

Query result

The query for assignee_id = 2 returned Create PostgreSQL schema and Replace mock data.

Catalog tables

information_schema listed three base tables and two views: member_task_view and project_progress_view.

Task columns

The tasks table contained task_id, project_id, title, status, assignee_id, due_date and priority.

Constraints

PostgreSQL reported primary-key, foreign-key, unique and check constraints for the three entities.

Indexes

The catalog showed the primary-key indexes, the unique email index and idx_tasks_assignee.

Important Catalog Output

```
information_schema.tables
projects                BASE TABLE
tasks                  BASE TABLE
users                  BASE TABLE
member_task_view       VIEW
project_progress_view  VIEW

pg_catalog.constraints
users_role_check       CHECK
users_pkey             PRIMARY KEY
users_email_key        UNIQUE
projects_owner_id_fkey FOREIGN KEY
projects_pkey          PRIMARY KEY
tasks_status_check     CHECK
tasks_project_id_fkey  FOREIGN KEY
tasks_assignee_id_fkey FOREIGN KEY
tasks_pkey             PRIMARY KEY

pg_indexes
projects_pkey
tasks_pkey
idx_tasks_assignee
users_pkey
```

users_email_key

Result

The Task Hub frontend mock data was successfully represented using the relational entities users, projects and tasks. The practical demonstrated the three-schema architecture through base tables, user-oriented views and PostgreSQL's physical storage information. Logical data independence was shown by adding a priority column without changing the existing view, while physical data independence was demonstrated by changing the query plan through an index without changing the SQL query itself. PostgreSQL's information_schema and pg_catalog were also used to inspect tables, columns, constraints and indexes.

Reference: Supplied Skilling 1 – RDBMS Foundations practical.